In [ ]:
```
import marimo as mo
```

In [ ]:
```
import numpy as np
import pandas as pd
import altair as alt
alt.data_transformers.disable_max_rows()
```

DataTransformerRegistry.enable('default')

Generalized Robust Policy Optimization (GRPO)

Executive Summary

GRPO (DeepSeekMath, 2024) is a reinforcement-learning fine-tuning algorithm designed to train LLMs on verifiable tasks (math, code, reasoning) **without a value/critic model**. Key ideas at a glance:

Idea	Detail
Group advantage estimation	Sample G outputs per prompt; normalize rewards within the group
No critic needed	Baseline comes from the group mean reward — no separate value network
PPO-style clipped objective	Surrogate loss with ϵ-clipping prevents large policy updates
KL regularizer	Penalizes deviation from a reference policy, maintaining stability
Empirical gains	GSM8K: PPO 82.9% → GRPO **88.2%**; MATH: 46.8% → **51.7%**

GRPO has since spawned several variants: **Dr. GRPO**, **DAPO**, **VAPO**, **Group-Robust PO**, and **Hybrid-GRPO**, each addressing different failure modes (entropy collapse, reward hacking, off-policy drift, group imbalance).

Learning Objectives

By the end of this notebook you will be able to:

1. **Explain** the motivation for GRPO and how it differs from PPO/RLHF pipelines.
2. **Derive** the group advantage estimator $A_i = (r_i - \bar{r})/(\sigma_r + \epsilon)$.
3. **Write out** the full GRPO surrogate objective with clipping and KL penalty.
4. **Identify** the role of each hyperparameter (G, ϵ, β, μ).

5. **Compare** GRPO against PPO, TRPO, DPO, and Group-Robust PO on key axes.
6. **Interpret** the empirical benchmark results from the DeepSeekMath paper.
7. **Describe** at least three known failure modes and how GRPO variants address them.
8. **Sketch** a training loop that applies GRPO to a new reasoning task.

Background & Prerequisites

Reinforcement Learning Basics

An RL agent selects actions a_t given state s_t under policy π_θ, receiving reward r_t. The goal is to maximise expected cumulative reward $J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_t r_t]$.

PPO & TRPO

TRPO (Schulman 2015) constrains the KL divergence between old and new policy to a trust region. **PPO** (Schulman 2017) approximates this via a clipped surrogate objective and a separate value network that estimates baselines. Both require a *critic* (value model) adding memory and compute overhead.

RLHF

Reinforcement Learning from Human Feedback (Ouyang 2022) trains a reward model from preference data, then runs PPO to align the LLM. The value model is a copy of the LLM — expensive at 7B+ parameters.

Robust RL

Standard RL optimises average-case reward. *Robust* RL seeks policies that perform well under worst-case or distributional perturbations — important when group-level fairness or tail-risk matters.

Mathematics Prerequisites

- Probability & expectations $\mathbb{E}[\cdot]$
- Policy gradient theorem: $\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot A]$
- KL divergence: $D_{\mathrm{KL}}(p||q) = \mathbb{E}_p[\log p/q]$
- Importance sampling ratio: $\rho = \pi_\theta / \pi_{\theta_{\text{old}}}$

Core Theory

Group Advantage Estimation

For each prompt q the algorithm samples a **group** of G output sequences $\{o_1, \ldots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$ and obtains scalar rewards $\{r_1, \ldots, r_G\}$. The advantage for output i is:

$$A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}, \qquad \bar{r} = \frac{1}{G}\sum_{j=1}^{G} r_j, \quad \sigma_r = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(r_j - \bar{r})^2}$$

where $\epsilon > 0$ is a small constant for numerical stability. This *within-group normalisation* acts as a natural baseline without any learned value network.

GRPO Objective

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q\sim\mathcal{D},\,\{o_i\}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G}\sum_{i=1}^{G} \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \Big(\min\big(\rho_{i,t}\,A_i,\ \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\big)$$

where $\rho_{i,t} = \dfrac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}$ is the per-token importance-sampling ratio and β is the KL penalty weight.

Critic-Free Design

PPO must train a value head $V_\phi(s_t)$ to estimate $\mathbb{E}[R_t|s_t]$ and compute GAE advantages. GRPO replaces this with the *group mean* — the average reward of sibling outputs for the same prompt. This eliminates:

- A second forward/backward pass for the critic.
- The memory footprint of a value-model copy.
- Bootstrapping errors from an imperfect critic.

Mathematical Derivation

Full GRPO Objective

Expanding the per-token notation, for a single prompt q the objective is:

$$J_{\text{GRPO}}(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \underbrace{\min\big(\rho_{i,t}A_i,\ \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)A_i\big)}_{\text{clipped surrogate}} - \underbrace{\beta\,\mathbb{E}_{o\sim\pi_\theta}\Big[\log}_{\text{KL per}}$$

Advantage for Outcome Supervision

With binary or scalar outcome rewards (e.g. correct/incorrect math answer):

$$A_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}, \qquad r_i \in \{0, 1\} \text{ (or continuous score)}$$

PPO Advantage (GAE) vs Group-Based

	PPO (GAE)	GRPO (Group-Based)
Baseline	Learned $V_\phi(s_t)$	Group mean $\bar{r}$
Temporal credit	Discounted multi-step TD	Whole-sequence scalar
Extra model	Yes (value head)	No
Variance	Lower (if critic accurate)	Controlled by group size G
Bias	From critic errors	From finite G

The GRPO derivation follows from the policy gradient theorem applied to the group-normalised rewards, with the clipping and KL terms providing the same monotonic-improvement guarantee as PPO's trust region.

PPO vs GRPO: Key Differences

In [ ]:
```
_comparison_data = pd.DataFrame({
    "Feature": [
        "Value Model", "Advantage Type", "Memory Overhead",
        "Reward Signal", "On-policy", "KL Constraint",
    ],
    "PPO": [
        "Required", "GAE (multi-step TD)", "High (2× LLM)",
        "Dense possible", "Yes", "Via penalty",
    ],
    "GRPO": [
        "Not needed", "Group-normalised", "Low (1× LLM)",
        "Outcome-level", "Yes", "Explicit β·D_KL",
    ],
})

_melted = _comparison_data.melt(id_vars="Feature", var_name="Algorithm", val

_chart = (
    alt.Chart(_melted)
    .mark_text(align="center", baseline="middle", fontSize=12)
    .encode(
        x=alt.X("Algorithm:N", axis=alt.Axis(labelFontSize=13, titleFontSize
        y=alt.Y("Feature:N", axis=alt.Axis(labelFontSize=12, titleFontSize=1
        text="Value:N",
        color=alt.Color(
            "Algorithm:N",
            scale=alt.Scale(range=["#4e79a7", "#f28e2b"]),
            legend=alt.Legend(title="Algorithm"),
        ),
    )
```

```
    .properties(
        title="PPO vs GRPO Feature Comparison",
        width=520,
        height=260,
    )
    .interactive()
)
ppo_grpo_comparison_chart = _chart
```

In [ ]:
```
ppo_grpo_comparison_chart
```

Surrogate Loss Clipping (Interactive)

In [ ]:
```
_rho = np.linspace(0.5, 1.5, 200)
_eps = 0.2
_clipped_rho = np.clip(_rho, 1 - _eps, 1 + _eps)

_advantage_specs = [(1.0, "A = +1 (positive)"), (-1.0, "A = -1 (negative)")]
_records = [
    {"rho": rho_val, "value": rho_val * A, "type": "Unclipped", "advantage":
    for rho_val, clipped_val in zip(_rho, _clipped_rho)
    for A, label in _advantage_specs
] + [
    {"rho": rho_val, "value": clipped_val * A, "type": "Clipped", "advantage
    for rho_val, clipped_val in zip(_rho, _clipped_rho)
    for A, label in _advantage_specs
]

_clip_df = pd.DataFrame(_records)

_region = pd.DataFrame({"x1": [0.8], "x2": [1.2]})
_region_band = (
    alt.Chart(_region)
    .mark_rect(opacity=0.08, color="gray")
    .encode(x=alt.X("x1:Q"), x2=alt.X2("x2:Q"))
)

_lines = (
    alt.Chart(_clip_df)
    .mark_line(strokeWidth=2)
    .encode(
        x=alt.X("rho:Q", title="Probability Ratio ρ"),
        y=alt.Y("value:Q", title="Surrogate Loss Contribution"),
        color=alt.Color(
            "advantage:N",
            scale=alt.Scale(range=["#e15759", "#4e79a7"]),
            legend=alt.Legend(title="Advantage"),
        ),
        strokeDash=alt.StrokeDash(
            "type:N",
            scale=alt.Scale(domain=["Unclipped", "Clipped"], range=[[1, 0],
            legend=alt.Legend(title="Line type"),
        ),
        tooltip=["rho:Q", "value:Q", "type:N", "advantage:N"],
```

```
    )
)

clipping_chart = (
    (_region_band + _lines)
    .properties(
        title="Surrogate Loss: Clipped vs Unclipped (ε = 0.2)",
        width=580,
        height=320,
    )
    .interactive()
)
```

In [ ]:
```
clipping_chart
```

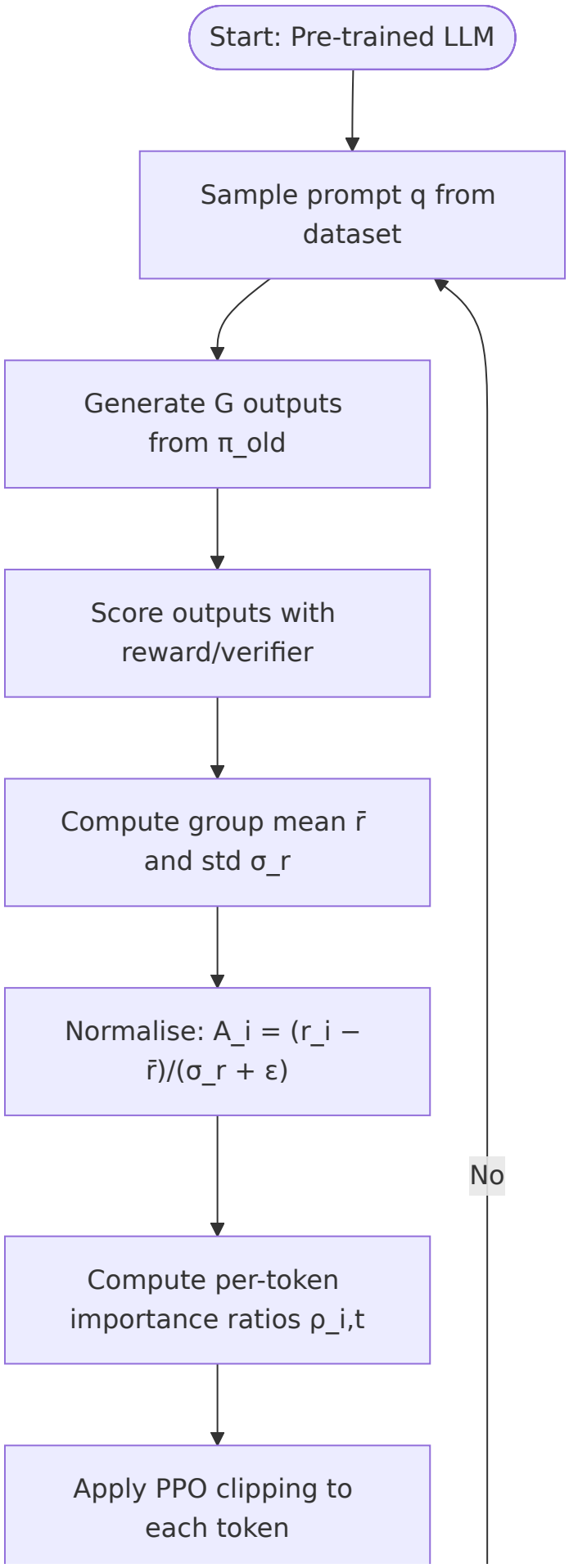
The **shaded band** marks the [1−ε, 1+ε] = [0.8, 1.2] region where clipping is inactive. Outside it the dashed line (clipped) flattens — removing the gradient signal that would push the policy too far from the old policy.

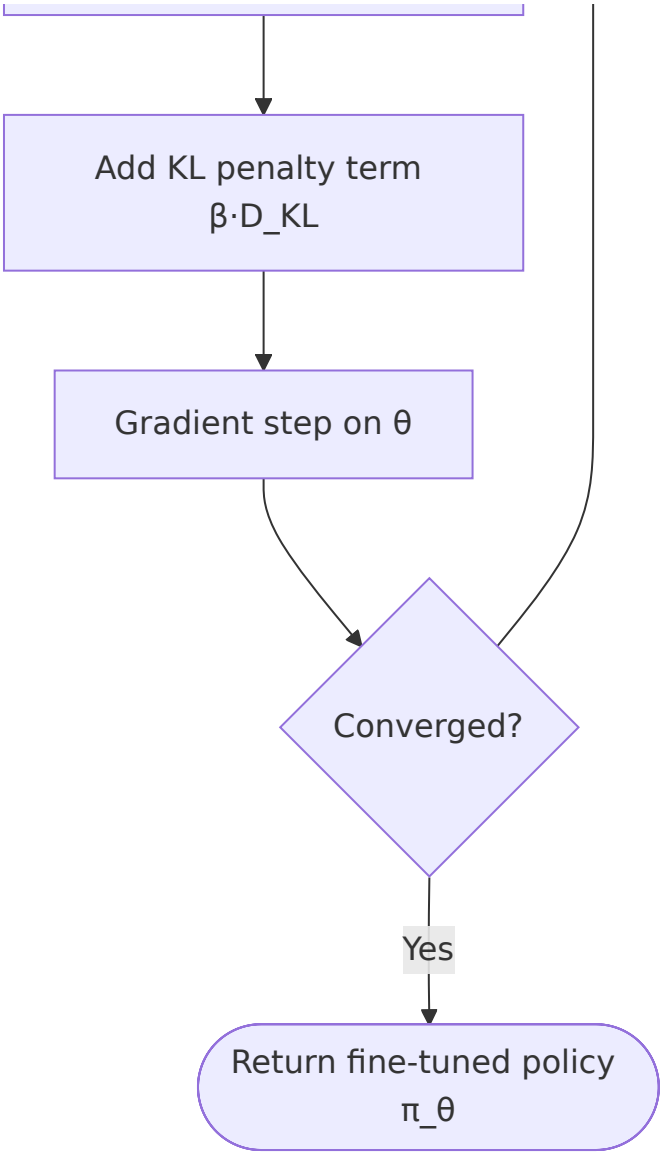
Algorithm Pseudocode

```
Algorithm: GRPO Fine-Tuning

Input : pre-trained LLM π_ref, dataset D, group size G,
        clip ε, KL weight β, learning rate η, steps T

Initialise π_θ ← π_ref

for step = 1 … T do
    Sample batch of prompts {q} from D
    for each prompt q do
        Sample G outputs {o_1,…,o_G} ~ π_θ_old(· | q)
        Compute rewards  {r_1,…,r_G}  (verifier / reward
model)
        r̄  ← mean({r_i}),  σ_r ← std({r_i})
        A_i ← (r_i − r̄) / (σ_r + ε_stab)     # group
advantage
    end for
    for each (q, o_i, A_i) do
        for each token t in o_i do
            ρ_{i,t} ← π_θ(o_{i,t}|q,o_{i,<t}) /
π_θ_old(o_{i,t}|q,o_{i,<t})
            L_{i,t} ← min(ρ_{i,t}·A_i,
clip(ρ_{i,t},1−ε,1+ε)·A_i)
        end for
    end for
    KL ← D_KL(π_θ ‖ π_ref)     # estimated over current batch
    J  ← mean_i mean_t L_{i,t} − β · KL
    θ  ← θ + η · ∇_θ J
    θ_old ← θ                   # update reference for next
step
```

```
    end for

    Return π_θ
```

GRPO Training Flowchart


Start: Pre-trained LLM
Sample prompt q from dataset
Generate G outputs from π_old
Score outputs with reward/verifier
Compute group mean r̄ and std σ_r
Normalise: A_i = (r_i − r̄)/(σ_r + ε)
Compute per-token importance ratios ρ_i,t
Apply PPO clipping to each token
No



Convergence Simulation: GRPO vs PPO

In [ ]:

```
def _logistic(x: np.ndarray, L: float, k: float, x0: float) -> np.ndarray:
    return L / (1.0 + np.exp(-k * (x - x0)))

_steps = np.arange(1, 101)
_rng = np.random.default_rng(42)

_grpo_acc = _logistic(_steps, L=88.5, k=0.08, x0=40) + _rng.normal(0, 0.6, 1
_ppo_acc = _logistic(_steps, L=83.0, k=0.06, x0=55) + _rng.normal(0, 0.6, 10

_grpo_acc = np.clip(_grpo_acc, 0, 100)
_ppo_acc = np.clip(_ppo_acc, 0, 100)

_conv_df = pd.DataFrame({
    "Step": np.tile(_steps, 2),
    "Accuracy (%)": np.concatenate([_grpo_acc, _ppo_acc]),
    "Algorithm": ["GRPO"] * 100 + ["PPO"] * 100,
})
```

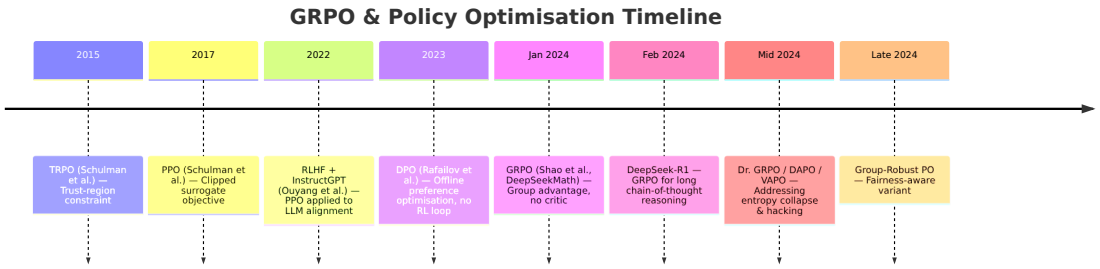
```
convergence_chart = (
    alt.Chart(_conv_df)
    .mark_line(point=False, strokeWidth=2)
    .encode(
        x=alt.X("Step:Q", title="Training Steps"),
        y=alt.Y("Accuracy (%):Q", title="Accuracy (%)", scale=alt.Scale(doma
        color=alt.Color(
            "Algorithm:N",
            scale=alt.Scale(domain=["GRPO", "PPO"], range=["#f28e2b", "#4e79
        ),
        tooltip=["Step:Q", "Accuracy (%):Q", "Algorithm:N"],
    )
    .properties(
        title="Simulated Convergence: GRPO vs PPO (synthetic data)",
        width=580,
        height=320,
    )
    .interactive()
)
```

In [ ]:
```
convergence_chart
```

Algorithm Comparison Table

Algorithm	Advantage Estimate	Value Critic?	On-policy?	Notes
PPO	GAE (multi-step TD)	✅ Yes	Yes	Stable; high memory
TRPO	GAE	✅ Yes	Yes	Exact KL constraint; slow
DPO	Implicit (preference pairs)	❌ No	No	Offline; no reward model needed
GRPO	Group-normalised scalar	❌ No	Yes	Efficient; outcome-supervised
Hybrid-GRPO	Group + partial TD	Partial	Yes	Balances bias/variance
RPO	Group + robust penalty	❌ No	Yes	Worst-group optimisation
RE-PO	Group + entropy bonus	❌ No	Yes	Prevents entropy collapse
Group-Robust PO	Min-group advantage	❌ No	Yes	CVaR-style fairness objective

Timeline: Key Milestones



Hyperparameter Guidance

Hyperparameter	Typical Range	Effect
Group size G	4 – 16	Larger G → lower variance advantage; more compute per step
Clip ϵ	0.1 – 0.3	Smaller → more conservative updates; larger → faster but risky
KL weight β	0.001 – 0.1	Larger → stays closer to reference policy; smaller → more exploration
Learning rate	1e-6 – 5e-6	Use warmup; too large causes instability with clipping
Optimisation steps per batch	1 – 4	> 1 risks off-policy drift since data is from $\pi_{\theta_{\text{old}}}$
Reward normalisation	Per-group	Must normalise *within group*, not globally, to preserve relative signal

Practical Tips

- Start with $G = 8$, $\epsilon = 0.2$, $\beta = 0.01$ as a baseline.
- If reward variance is very low (all-correct or all-wrong groups) increase G.
- If the model diverges from the reference too quickly, increase β.
- Monitor token-level entropy; a sharp drop signals entropy collapse → use Dr. GRPO or RE-PO.
- For math/code tasks with a symbolic verifier the KL term may be reduced since the verifier already provides clean reward signal.

Empirical Results: DeepSeekMath Benchmarks

In [ ]:
```
_results_df = pd.DataFrame({
    "Task": ["GSM8K", "GSM8K", "MATH", "MATH", "CMATH", "CMATH"],
    "Method": ["PPO", "GRPO", "PPO", "GRPO", "PPO", "GRPO"],
    "Accuracy (%)": [82.9, 88.2, 46.8, 51.7, 84.6, 88.8],
})

_bar = (
```

```
    alt.Chart(_results_df)
    .mark_bar()
    .encode(
        x=alt.X("Task:N", title=None, axis=alt.Axis(labelFontSize=13)),
        xOffset=alt.XOffset("Method:N"),
        y=alt.Y(
            "Accuracy (%):Q",
            title="Accuracy (%)",
            scale=alt.Scale(domain=[40, 95]),
        ),
        color=alt.Color(
            "Method:N",
            scale=alt.Scale(domain=["PPO", "GRPO"], range=["#4e79a7", "#f28e
            legend=alt.Legend(title="Method"),
        ),
        tooltip=["Task:N", "Method:N", alt.Tooltip("Accuracy (%):Q", format=
    )
)

_text = (
    alt.Chart(_results_df)
    .mark_text(dy=-6, fontSize=11, fontWeight="bold")
    .encode(
        x=alt.X("Task:N"),
        xOffset=alt.XOffset("Method:N"),
        y=alt.Y("Accuracy (%):Q"),
        text=alt.Text("Accuracy (%):Q", format=".1f"),
        color=alt.Color("Method:N", scale=alt.Scale(domain=["PPO", "GRPO"],
    )
)

empirical_chart = (
    (_bar + _text)
    .properties(
        title="GRPO vs PPO — DeepSeekMath Benchmarks",
        width=480,
        height=340,
    )
    .interactive()
)
```

In [ ]:
```
empirical_chart
```

Empirical Results Table

Results from DeepSeekMath (Shao et al., 2024) on the 7B model:

Benchmark	SFT Baseline	PPO	GRPO	GRPO Δ vs PPO
GSM8K	74.8%	82.9%	**88.2%**	+5.3 pp
MATH	35.9%	46.8%	**51.7%**	+4.9 pp
CMATH	76.6%	84.6%	**88.8%**	+4.2 pp

Benchmark	SFT Baseline	PPO	GRPO	GRPO Δ vs PPO
OCW Courses	12.8%	15.1%	**16.6%**	+1.5 pp

GRPO consistently outperforms PPO across all math benchmarks while using **half the GPU memory** (no value model) and achieving faster wall-clock training time.

Open Problems & Future Directions

Known Failure Modes

1. **Entropy Collapse** — Extended GRPO training can reduce output diversity, causing the model to repeat a narrow set of solution patterns. *Mitigation*: RE-PO entropy bonus; DAPO entropy-regularised objective.
2. **Reward Hacking** — With weak verifiers the model can exploit loopholes (e.g. formatting tricks) rather than learning genuine reasoning. *Mitigation*: Robust reward shaping; process reward models (PRMs).
3. **Off-Policy Drift** — Multiple gradient steps per sample accumulate importance-sampling error. *Mitigation*: Limit to 1–2 steps; re-sample frequently.
4. **Group Homogeneity** — If all G outputs receive identical rewards ($\sigma_r \approx 0$) the advantage is ill-defined and the update is zero. *Mitigation*: Adaptive G; temperature annealing to encourage diversity.

Future Directions

- **Process-level GRPO**: Reward intermediate reasoning steps (PRMs) rather than only final answers to provide denser signal.
- **Multi-turn GRPO**: Extend to dialogue settings with session-level rewards.
- **Continual GRPO**: Online adaptation where the dataset itself grows from model interactions.
- **Theoretical analysis**: Tight convergence bounds for the critic-free group estimator vs GAE; characterising the bias-variance frontier as G varies.
- **Fairness-aware variants**: Group-Robust PO minimises worst-group error, relevant for demographic-fairness in alignment.

Further Reading

Resource	Type	Notes
Shao et al. (2024) — *DeepSeekMath*	Paper	Original GRPO paper; math RL fine-tuning

Resource	Type	Notes
DeepSeek-R1 Technical Report (2025)	Report	GRPO for long CoT reasoning at scale
Schulman et al. (2017) — *PPO*	Paper	Foundational clipped surrogate objective
Schulman et al. (2015) — *TRPO*	Paper	Trust-region policy optimisation
Rafailov et al. (2023) — *DPO*	Paper	Offline preference optimisation baseline
Ouyang et al. (2022) — *InstructGPT*	Paper	RLHF pipeline with PPO
Dr. GRPO (2024)	Paper	Addresses entropy collapse in GRPO
DAPO (2025)	Paper	Dynamic sampling + entropy regularisation
Group-Robust PO (2024)	Paper	CVaR-style fairness-aware variant
Hugging Face TRL docs	Docs	Open-source GRPO trainer implementation